Sanaul Haque

Michael_Wang_Proposal

Assignment 108

Document Details

Submission ID
trn:oid:::3117:636700942

Submission Date
Oct 9, 2026, 11:24 PM GMT+6

Download Date
Oct 9, 2026, 11:26 PM GMT+6

File Name
Michael_Wang_Proposal.pdf

File Size
624.0 KB

7 Pages

2,976 Words

16,646 Characters

15% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

14 Not Cited or Quoted 12%
Matches with neither in-text citation nor quotation marks

4 Missing Quotations 1%
Matches that are still very similar to source material

3 Missing Citation 2%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

14% Internet sources
13% Publications
14% Submitted works (Student Papers)

Match Groups

- **14** Not Cited or Quoted 12%
  Matches with neither in-text citation nor quotation marks
- **4** Missing Quotations 1%
  Matches that are still very similar to source material
- **3** Missing Citation 2%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

14% Internet sources
13% Publications
14% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	arxiv.org 4%
2	Student papers	University of Leicester LTI on 2026-07-23 2%
3	Student papers	Higher Education Commission Pakistan on 2026-08-09 2%
4	Student papers	Texas A&M University - Commerce on 2026-02-15 1%
5	Publication	Gautam Krishna Mooppil, Abhijai Sasikumar, Abeer Mathur, Subramaniyaswamy ... 1%
6	Student papers	University of Canberra on 2025-10-27 1%
7	Student papers	University of New York Tirana on 2026-07-20 <1%
8	Internet	s3-eu-west-1.amazonaws.com <1%
9	Student papers	Thesis 2026 on 2026-04-17 <1%
10	Publication	Wei Tao, Xiaoyang Qu, Peiqiang Wang, Guokuan Li, Jiguang Wan, Kai Lu, Jianzong ... <1%

11 Internet

research-repository.st-andrews.ac.uk <1%

12 Internet

lutpub.lut.fi <1%

13 Internet

scholar.xjtlu.edu.cn <1%

14 Student papers

Tilburg University on 2026-05-21 <1%

Full Name: Michael Wang

Institution Name: Nikola Tesla STEM High School

Country: United States

Category: Junior

Class/Year: Senior/4th year

Major/Subjects: Computer Science

Research Topic: Artificial Intelligence

Registered Email Address: bear65160@gmail.com

Submission Date: October 4th, 2026

MetaKV: Prompt-Adaptive Selection of KV Cache Compression Methods for Resource-Constrained LLM Inference

Abstract

Compressing the key-value (KV) cache can lower the memory required for large language model (LLM) inference, especially in long-context inputs. However, different KV cache compression methods have trade-offs among accuracy, end-to-end latency and peak KV cache memory utilization (including quantization, token eviction, and sparse attention). Consequently, a single fixed method may not be optimal under all prompt and deployment environments. This study introduces MetaKV, a pipeline that adaptively selects a KV cache compression configuration based on a user prompt and user-specified latency and peak memory thresholds. The method pool contains 10 configurations that include KVQuant, H_2O, and RocketKV and three compression settings each, together with an uncompressed 16-bit floating-point precision (FP16) baseline. MetaKV uses lightweight models to predict each candidate configuration's end-to-end latency, peak memory, and likelihood of producing a correct answer. Using these predictions, MetaKV filters out the ineligible configurations and selects the eligible configuration with the highest predicted accuracy. MetaKV will be tested using the Llama-3.1-8B LLM and four datasets: GSM8K (math), HellaSwag (common sense reasoning), ARC-Challenge (science), and SQuAD 1.1 (reading comprehension). Its performance will be evaluated across different latency-memory constraint pairs and with a metric called the Constrained Success Rate (CSR), representing the proportion of prompts that MetaKV answers correctly while satisfying both latency and memory constraints. MetaKV's CSR will be compared to that of a static baseline, representing the best single configuration the user could commit to in advance, and an oracle, MetaKV's performance upper bound. McNemar's test will be used to determine if the difference in CSR between MetaKV and the static baseline is statistically significant. It is hypothesized that MetaKV will consistently outperform the static baseline in terms of CSR and the difference will be statistically significant under varying values for the latency and peak memory thresholds.

Introduction

Large Language Models (LLMs) have increasingly been used in the workplace through writing reports, code generation [1], and decision making. As LLM usage grows, reducing the computational and GPU-memory costs of inference has become increasingly important [2]–[4]. As chatbots, LLMs need to store conversation history using a Key-Value (KV) cache which takes up around 80% of total GPU memory during long-context LLM inference [2]–[4]. Prior work has introduced methods to compress the KV cache and reduce memory overhead, allowing LLMs to run in environments with limited GPU resources [2]–[4].

The different lossy KV cache compression methods utilize different approaches to compress the KV Cache and can be divided into three major categories: (i) Quantization stores the values in the KV Cache using fewer bits, (ii) token eviction permanently discards unimportant tokens from the KV Cache, and (iii) sparse attention only uses a portion of cached tokens during each decode step [2]–[4]. Since these compression methods utilize different algorithms to shrink the KV Cache, their relative performance can vary in terms of latency, peak memory, and accuracy [2]–[4]. For example, the steps in quantization including quantizing and dequantizing result in a higher latency than other methods [2]. However, its conversion from Floating-Point full 16-bit precision (FP16) storage to using only a few bits results in the least memory [2]. Token eviction, on the other hand, has lower latency due to its fewer computations but does not conserve as

much memory as quantization [2], [3]. Here, there is a clear tradeoff between memory and latency while using different compression methods [2]–[4].

The relative importance of accuracy, latency, and peak memory depends on the model's deployment environment. For example, smartphones and other edge devices provide limited GPU memory, requiring low memory [5]. Here, quantization's low memory would be the most suitable [2]. On the other hand, there are cases where latency is the bottleneck. For example, customer-service chatbots are interactive applications, so they require fast responses for user-friendliness [6]. In this case, token eviction will be more suitable due to its lower latency [3]. However, if the user wanted to maximize the LLM's response accuracy but had no limit on memory or latency, having no compression method and retaining the entire KV cache would be optimal [2]–[4]. These scenarios show that there is no single method that is best under all circumstances [2]–[4].

Despite advances in individual KV cache compression methods, there remains a significant research gap: existing work does not address how the most suitable compression method could be selected based on the user's prompt and the LLM's deployment environment [2]–[4]. This research gap motivates MetaKV, a framework that adaptively selects from a pool of ten method configurations based on the input prompt and user-specified latency and peak memory constraints.

Literature Review

Prior work has proposed KV cache compression methods that can be categorized as lossy and lossless. Lossless methods reduce memory overhead while keeping original KV cache values, preserving the accuracy of the LLM. On the other hand, lossy approaches obtain larger memory savings by removing information or lowering its precision, although these changes can decrease answer accuracy [2]–[4]. Because larger memory reduction becomes important for long-context inputs [2]–[4], this study examines different lossy compression methods. KVQuant, H_2O, and RocketKV are the compression methods focused on in this study because they are specific proposed algorithms that utilize the three major categories of lossy KV cache compression: quantization, token eviction, and sparse attention [2]–[4]. Three compression settings will be used for each method along with an uncompressed FP16 baseline, totaling 10 configurations for MetaKV to select from. The uncompressed FP16 baseline means that no compression method is applied, retaining the entire KV cache and its precision. These different configurations exhibit a set of trade-offs among accuracy, latency, and peak memory [2]–[4].

KVQuant is a KV cache compression method that incorporates quantization with other techniques to minimize the resulting accuracy loss. Because outlier values in the KV cache are concentrated in consistent channels, it uses per-channel quantization, allowing each channel to use its own scaling parameters and thereby reducing quantization error. Additionally, it stores extreme outliers in full FP16 precision to avoid the large errors that result from quantizing them [2]. This study evaluates KVQuant using 2-bit, 3-bit, and 4-bit precision levels when storing the values in the KV cache.

H_2O uses a different approach to compressing the KV cache by removing entire tokens that are unimportant instead of reducing precision. To measure how important a token is, it calculates an accumulated attention score which represents how much attention a certain token received from subsequent tokens. H_2O includes a budget for the maximum number of tokens that the KV cache can store. If the KV cache exceeds the budget, H_2O evicts older tokens with the lowest

accumulated attention scores [3]. The evaluated H_2O settings retain 20%, 40%, and 60% of the tokens in the KV cache.

RocketKV combines token pruning with sparse attention. RocketKV first uses attention from an observation window near the end of the prompt to calculate aggregated attention scores for earlier tokens and retain those with the highest scores. In its second stage, Hybrid Sparse Attention uses only a small subset of tokens from the KV cache during LLM inference [4]. RocketKV is evaluated at $8\times$, $16\times$, and $32\times$ compression, meaning that the KV cache is reduced to approximately $1/8$, $1/16$, and $1/32$ of its original size [4].

Research Question

This study aims to answer the following central research question: Under predefined end-to-end latency and peak KV-cache memory constraints, can a prompt-adaptive selection framework among ten KV-cache compression configurations achieve a higher Constrained Success Rate than the best-performing static configuration?

Hypothesis

Under different constraint pairings for end-to-end latency and peak memory, MetaKV can consistently achieve a higher Constrained Success Rate than the best static configuration because its prompt-adaptive approach can select the most suitable method configuration for each individual prompt.

Methodology

The MetaKV pipeline first requires an offline profiling stage. To do this, MetaKV takes a random sample of 2,048 prompts from each of four datasets that represent a diverse range of topics: GSM8K [7] (math), SQuAD 1.1 [8] (reading comprehension), HellaSwag [9] (common sense), and ARC-Challenge [10] (scientific reasoning). This results in a total of 8,192 prompts for MetaKV. Llama-3.1-8B [11] will process all 8,192 prompts under each of the 10 method configurations on an NVIDIA RTX PRO 6000 Blackwell Server Edition GPU with 96 GB of VRAM on Google Colab. For each prompt-configuration pair, the end-to-end latency, peak memory, and response correctness will be recorded. For generative tasks like GSM8K and SQuAD 1.1, the LLM will be given a set of instructions on how to format its response so that the final answer can be extracted and compared with the ground truth to determine correctness. For multiple-choice datasets like ARC-Challenge and HellaSwag, the answer choice with the lowest negative log-likelihood is selected and compared to the correct answer.

After collecting data for the correctness, end-to-end latency, and peak memory using all 10 configurations across all 8,192 prompts, 85% of the sample prompts will be used for offline profiling. In this stage, this subsample of prompts will be used to train three Light Gradient Boosting Machines (LightGBMs) [12] using repeated five-fold cross-validation. Ten lightweight features, including token length and token diversity, are extracted from each prompt and inputted into the LightGBMs that predict the end-to-end latency, peak memory, and probability of a correct response under each configuration. Thus, the three LightGBMs each produce 10 outputs.

After the offline profiling stage, the MetaKV pipeline is tested using the remaining 15% of the prompts. The pipeline will be tested across a grid of latency and peak memory constraints. The latency limits are defined as $L_{max} \in \{2, 5, 10, 50\}$ seconds, and the peak memory limits are defined as $M_{max} \in \{5, 10, 30, 80\}$ MB. Every possible $(L_{max}, M_{\max})$ will be tested, resulting in

$4 \times 4 = 16$ constraint pairs. The MetaKV pipeline will be run on each prompt in the test set using each constraint pair.

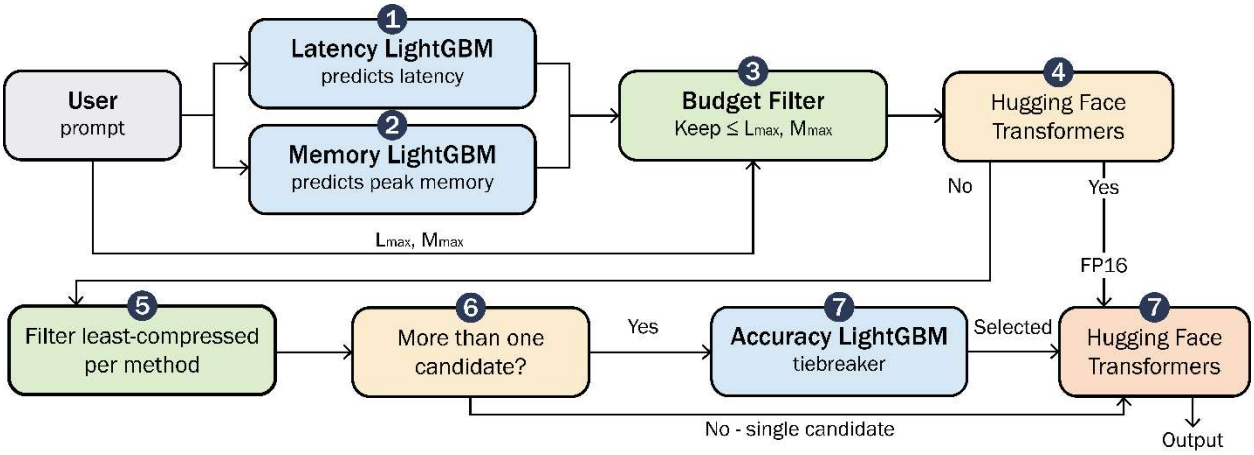


Figure 1: Overview of the MetaKV selection pipeline. Given a user prompt and the specified latency and peak-memory constraints, MetaKV picks the method configuration predicted to be the most suitable.

Figure 1 shows that the MetaKV pipeline first inputs the 10 extracted features of the initial prompt into the latency and peak memory LightGBMs, as shown in steps 1 and 2, to predict the latency and peak memory using each of the 10 configurations. Step 3 shows a budget filter that retains only configurations whose predicted latency and peak memory satisfy the user's constraints (L_{max} and M_{max}). As shown in step 4, if the uncompressed FP16 configuration is among the chosen configuration candidates, then it is immediately chosen because it keeps the entire KV cache, resulting in the highest chance of an accurate response [2]–[4]. If FP16 is not among the candidates, only the least aggressive remaining setting from each compression method is kept as shown in step 5. If there is only one candidate configuration left, as shown in step 6, it is selected. Otherwise, the prompt's features are inputted into the accuracy LightGBM as a final tiebreaker, and the candidate configuration with the highest predicted probability of answering the prompt correctly is selected as shown in step 7. Finally, the LLM uses the selected method configuration to generate a response for the user with Hugging Face Transformers [13].

If the budget filter in Step 3 rejects every configuration, MetaKV calculates a violation score (v_c) for each configuration given by: $v_c = \frac{\hat{L}_c}{L_{\max}} + \frac{\hat{M}_c}{M_{\max}}$, where $\hat{L}_c$ and $\hat{M}_c$ are the predicted end-to-end latency and peak memory utilization for configuration c. The configuration with the lowest v_c is selected for that prompt.

Evaluation

To evaluate MetaKV's performance, this study introduces a metric called the Constrained Success Rate (CSR), which captures MetaKV's ability to balance performance and efficiency. CSR measures the proportion of test prompts answered correctly without exceeding the user's constraints: $CSR = \frac{1}{N}\sum_{i=1}^{N} I(correct_i \wedge \ell_i \leq L_{max} \wedge m_i \leq M_{max})$

MetaKV's CSR will be compared to that of two baselines. To evaluate the benefit of MetaKV's adaptive approach, it will be compared to a static baseline, which uses a single configuration across all test prompts. For each constraint pair, the CSR of every fixed configuration will be

calculated on the training set. The configuration with the highest training-set value will serve as the static baseline and will be evaluated on the test set alongside MetaKV. If MetaKV's CSR consistently outperforms that of the static baseline across different constraint pairs, this provides evidence that MetaKV's adaptive approach is more effective than using one fixed configuration.

To determine if the difference in CSR between MetaKV and the static baseline is statistically significant, McNemar's test [14] will be applied to each of the 16 constraint pairs. For each test prompt, MetaKV and the static baseline produce a binary outcome of success or failure, and McNemar's test evaluates whether the prompts where outcomes between MetaKV and the static baseline differ favor one method more than chance would predict. A Holm–Bonferroni correction [15] will be applied across the 16 tests at a significance level of 0.05. MetaKV is considered to have a significant improvement in CSR for a constraint pair if the Holm–Bonferroni-adjusted p-value is below the significance level with MetaKV having a greater CSR.

MetaKV's CSR will also be compared to that of an oracle, which represents the upper bound of MetaKV's performance. To calculate the oracle's CSR for each constraint pair, the actual latency and peak memory for each prompt will be used to filter the configurations that satisfy both constraints. If any candidate answers the prompt correctly, the oracle counts that prompt as a success. Comparing MetaKV's and the oracle's CSR shows how close MetaKV comes to its best possible performance. A small gap indicates the LightGBM predictors have sufficient accuracy while a larger gap indicates prediction and selection errors that leave room for future work.

Potential Limitations

- **Evaluation Scope:** This study utilizes one LLM, one GPU, and four datasets to test MetaKV's performance, so the trained LightGBMs are specific to this single experimental setup. Testing MetaKV on other LLMs or hardware platforms will require re-running offline profiling to retrain the LightGBM predictors.
- **Prediction Accuracy:** MetaKV relies on the accuracy of LightGBMs, and latency is harder to predict because the number of output tokens is not known beforehand. An underestimate of latency or peak memory could cause MetaKV to select a configuration that violates the user's constraints while an overestimate could eliminate a configuration that would have satisfied the constraints.
- **Fixed Selection Rules:** MetaKV's configuration selection rules assume that lower KV cache compression settings always produce better accuracy since it retains more of the KV cache. However, this assumption may not always hold. Future work could directly utilize the accuracy LightGBM to select a configuration out of the valid candidates if its accuracy could be significantly improved.

Timeline

- Weeks 1–2: Literature review finalization and experimental design
- Weeks 3–4: Implementation and testing of the 10 compression configurations
- Weeks 5–7: Prompt profiling and data collection
- Week 8: LightGBM training and cross-validation
- Week 9: MetaKV evaluation against the static baseline and oracle
- Week 10: Results analysis and visualization

References

[1] Y. Dong et al., “A survey on code generation with LLM-based agents,” arXiv:2508.00083, 2025.

[2] C. Hooper et al., “KVQuant: Towards 10 million context length LLM inference with KV cache quantization,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[3] Z. Zhang et al., “H2O: Heavy-hitter oracle for efficient generative inference of large language models,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[4] P. Behnam, Y. Fu, R. Zhao, P.-A. Tsai, Z. Yu, and A. Tumanov, “RocketKV: Accelerating long-context LLM inference via two-stage KV cache compression,” in *Proc. 42nd Int. Conf. Machine Learning*, ser. Proc. Machine Learning Research, vol. 267, pp. 3358–3392, 2025.

[5] Z. Xue, Y. Song, Z. Mi, L. Chen, Y. Xia, and H. Chen, "PowerInfer-2: Fast large language model inference on a smartphone," arXiv:2406.06282, 2024.

[6] U. Gnewuch, S. Morana, M. T. P. Adam, and A. Maedche, “Opposing effects of response time in human–chatbot interaction: The moderating role of prior experience,” *Bus. Inf. Syst. Eng.*, vol. 64, no. 6, pp. 773–791, Dec. 2022, doi: 10.1007/s12599-022-00755-x.

[7] K. Cobbe et al., “Training verifiers to solve math word problems,” arXiv:2110.14168, 2021.

[8] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “SQuAD: 100,000+ questions for machine comprehension of text,” in *Proc. 2016 Conf. Empirical Methods in Natural Language Processing*, Austin, TX, USA, Nov. 2016, pp. 2383–2392, doi: 10.18653/v1/D16-1264.

[9] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, “HellaSwag: Can a machine really finish your sentence?,” in *Proc. 57th Annu. Meeting Assoc. Comput. Linguistics*, Florence, Italy, Jul. 2019, pp. 4791–4800, doi: 10.18653/v1/P19-1472.

[10] P. Clark et al., “Think you have solved question answering? Try ARC, the AI2 reasoning challenge,” arXiv:1803.05457, 2018.

[11] A. Grattafiori et al., “The Llama 3 herd of models,” arXiv:2407.21783, 2024.

[12] G. Ke et al., “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems*, vol. 30, pp. 3149–3157, 2017.

[13] T. Wolf et al., “Transformers: State-of-the-art natural language processing,” in Proc. 2020 Conf. Empirical Methods in Natural Language Processing: System Demonstrations, Online, Oct. 2020, pp. 38–45, doi: 10.18653/v1/2020.emnlp-demos.6.

[14] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," Psychometrika, vol. 12, no. 2, pp. 153–157, 1947, doi: 10.1007/BF02295996.

[15] S. Holm, "A simple sequentially rejective multiple test procedure," Scandinavian Journal of Statistics, vol. 6, no. 2, pp. 65–70, 1979.